Intro to multithreaded programming with POSIX threads

Main resource: the content and the images are copied from https://hpc-tutorials.llnl.gov/posix/

Concurrency & Parallel computing

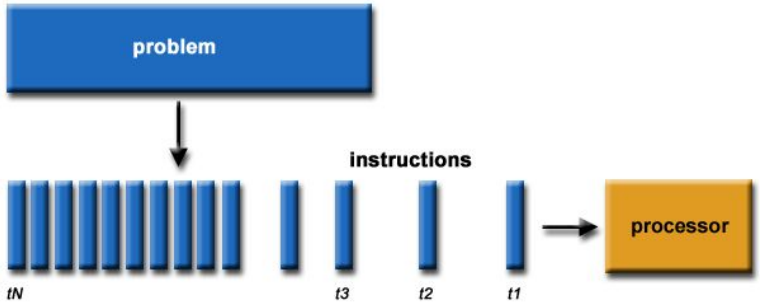
Operations are **concurrent** if they can progress independently at the same time

A process is working on multiple tasks at the same time

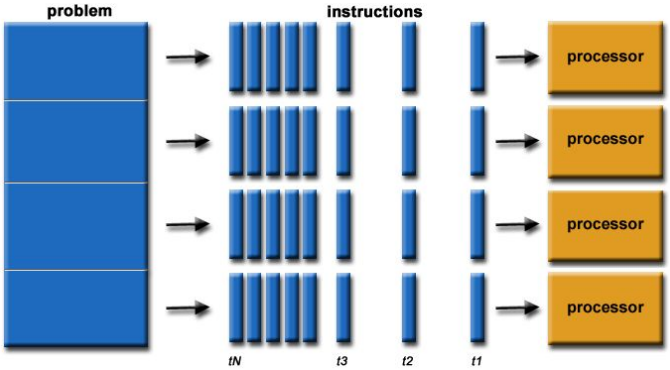
Parallelism: splitting tasks into subtasks

Parallel computing: running subtasks (concurrent parts) parallel in multiple processor

Serial computing



Parallel computing



Matrix addition: Concurrent parts?

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

Data: **a_ij, b_ij**

Task: Addition

The same task, different data!

Matrix multiplication: Concurrent parts?

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

C = AB

$$\mathbf{C} = \begin{pmatrix} a_{11}b_{11} + \cdots + a_{1n}b_{n1} & a_{11}b_{12} + \cdots + a_{1n}b_{n2} & \cdots & a_{11}b_{1p} + \cdots + a_{1n}b_{np} \\ a_{21}b_{11} + \cdots + a_{2n}b_{n1} & a_{21}b_{12} + \cdots + a_{2n}b_{n2} & \cdots & a_{21}b_{1p} + \cdots + a_{2n}b_{np} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{11} + \cdots + a_{mn}b_{n1} & a_{m1}b_{12} + \cdots + a_{mn}b_{n2} & \cdots & a_{m1}b_{1p} + \cdots + a_{mn}b_{np} \end{pmatrix}$$

Data: **a_i, B** or **A, bj**

Task: Multiplication(and addition)

The same task, different data!

What if we want to do both?

Matrix addition

$\mathbf{T} = \mathbf{A} + \mathbf{B}$

Matrix multiplication

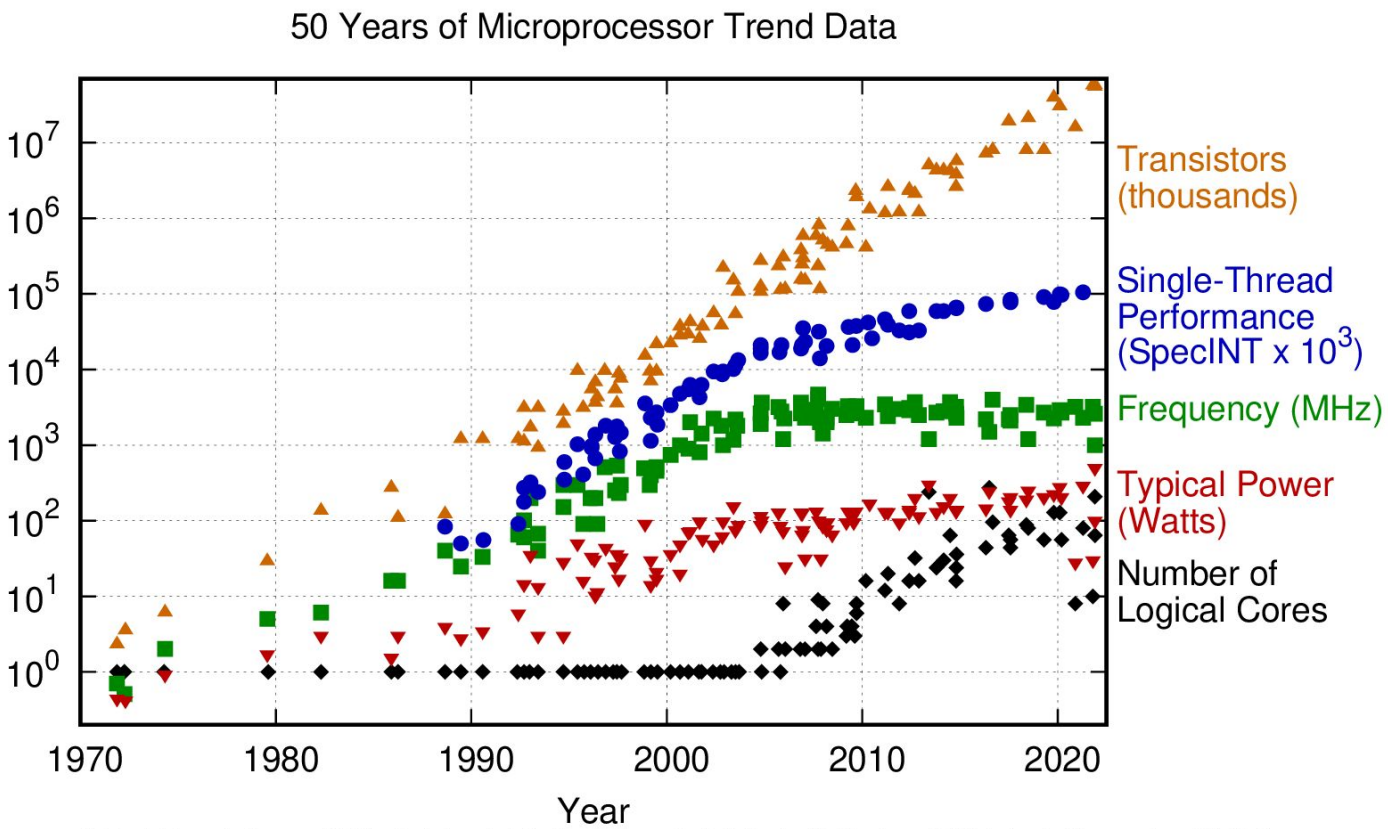
$\mathbf{C} = \mathbf{AB}$

Data: $\mathbf{A}, \mathbf{B}$

Task: Addition, Multiplication

The same data, different concurrent tasks!

Moore's law



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

What is a thread (software thread)?

Like processes, threads allow an application to perform multiple tasks concurrently

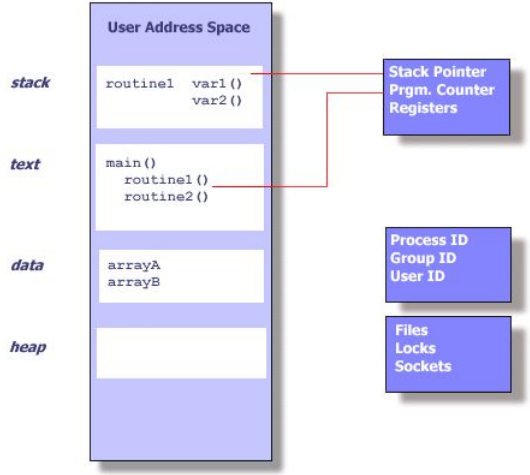
A thread is an independent stream of instructions that can be scheduled to run by OS

- Threads can be scheduled to use the CPU in any order with any other thread
  - Threads act concurrently
  - Threads can be executed in parallel in a multicore system
- suspending execution of one thread does not cause the suspension of the other
- If the order that threads execute matters, then threads must be synchronized
- To the software developer, the concept of a "procedure" that runs independently from its main program

Processes vs Threads

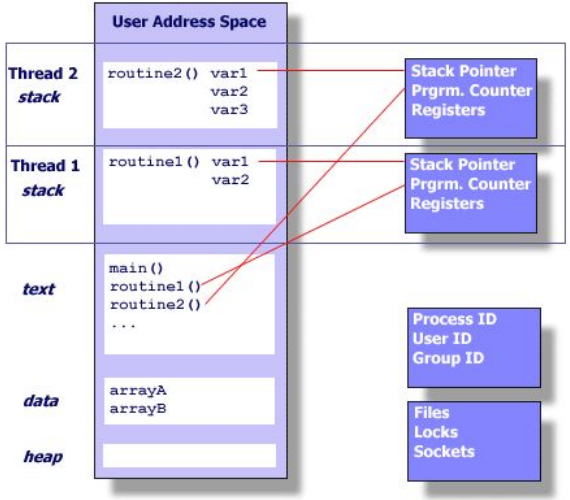
Process

- Each process has its own virtual address space
- fork() is expensive



Thread maintains only its own

- Stack pointer
- Registers
- Scheduling properties (such as policy or priority)
- Set of pending and blocked signals
- Thread specific data.



Why threads over processes?

Creating a new process can be expensive

Time

- A call into the operating system is needed
- Context-switching involves the operating system

Memory

- The entire process must be replicated
- The cost of inter-process communication and synchronization of shared data
    - May involve calls into the operating system kernel

- Threads can be created without replicating an entire process
- Creating a thread is done in user space rather than kernel
- Shared virtual address space
- communications between two threads is usually faster and easier to implement then communications between two processes.

Timing of 50,000 process/thread creations

Platform	fork()			pthread_create()		
	real	user	sys	real	user	sys
Intel 2.6 GHz Xeon E5-2670 (16 cores/node)	8.1	0.1	2.9	0.9	0.2	0.3
Intel 2.8 GHz Xeon 5660 (12 cores/node)	4.4	0.4	4.3	0.7	0.2	0.5
AMD 2.3 GHz Opteron (16 cores/node)	12.5	1.0	12.5	1.2	0.2	1.3
AMD 2.4 GHz Opteron (8 cores/node)	17.6	2.2	15.7	1.4	0.3	1.3
IBM 4.0 GHz POWER6 (8 cpus/node)	9.5	0.6	8.8	1.6	0.1	0.4
IBM 1.9 GHz POWER5 p5-575 (8 cpus/node)	64.2	30.7	27.6	1.7	0.6	1.1
IBM 1.5 GHz POWER4 (8 cpus/node)	104.5	48.6	47.2	2.1	1.0	1.5
INTEL 2.4 GHz Xeon (2 cpus/node)	54.9	1.5	20.8	1.6	0.7	0.9
INTEL 1.4 GHz Itanium2 (4 cpus/node)	54.5	1.1	22.2	2.0	1.2	0.6

Problems in threaded programs

Application is less robust; data sharing requires synchronization to avoid race conditions

If a thread misbehaves, it can corrupt data of other threads within same process

If a thread crashes, the entire process crashes

Pthreads API

- Historically, hardware vendors have implemented their own proprietary versions of threads.
- In order to take full advantage of the capabilities provided by threads, a standardized programming interface was required.
- For UNIX systems, a standardized C language threads programming interface has been specified by the IEEE POSIX 1003.1c standard.
- Implementations that adhere to this standard are referred to as POSIX threads, or Pthreads.
- Most hardware vendors now offer Pthreads in addition to their proprietary API's.

- Pthreads are implemented with
  - a **pthread.h** header/include file
  - and a thread library
- All identifiers in the threads library begin with **pthread_.**

- **Compiler command**

gcc -pthread	GNU C
g++ -pthread	GNU C++

- The Pthreads API can be informally grouped into four major groups:
  - **(today)Thread management:**
    - Routines that work directly on threads - creating, detaching, joining, etc.
  - **Mutexes:** Routines that deal with synchronization, called a "mutex", which is an abbreviation for "mutual exclusion". Mutex functions provide for creating, destroying, locking and unlocking mutexes.
  - **Condition variables:** Routines that address communications between threads that share a mutex. Based upon programmer specified conditions.
  - **Synchronization:** Routines that manage read/write locks and barriers.

Creating a thread

```
#include <pthread.h>
int pthread_create(pthread_t *thread,
        const pthread_attr_t *attr,
        void *(*start_routine)(void*),
        void *arg);
```

- **On success returns 0**
- **On error, it returns error code**

pthread_create creates a new thread and makes it executable.

This routine can be called any number of times from anywhere within your code.

pthread_create arguments:

- `thread`: A unique identifier for the new thread returned by the subroutine.
- `attr`: You can specify a thread attributes object, or NULL for the default values.
- `start_routine`: The new thread starts execution by invoking this function
- `arg`: A single argument that may be passed to `start_routine`.
  - It must be passed by reference as a pointer cast of type void.
  - NULL may be used if no argument is to be passed.

Creating a thread

```
#include <pthread.h>
int pthread_create(pthread_t *thread,
       const pthread_attr_t *attr,
       void *(*start_routine)(void*),
       void *arg);
```

- On success returns 0
- On error, it returns error code

```
/*example*/
pthread_t thread;
int r = pthread_create(&thread, NULL, func, &arg);
if (r != 0)
  perror("pthread_create");
```

Thread termination

The execution of a thread terminates in one of the following ways:

- The thread's start function performs a return specifying a return value for the thread.
  - `return 0;//in func()`
- The thread calls `pthread_exit()`
  - `void pthread_exit(void *retval);`
    - Terminates the calling thread
    - `retval` specifies the return value of the thread
    - This value is available to another thread in the same process that calls **`pthread_join()`**
    - `retval` **should point to the heap not to the stack!**

Process termination

A multithreaded process terminates when:

- one of its threads calls `exit();`
- the main thread (`main()` function) performs **`return`**
- it receives a termination signal
- all threads have called pthread_exit()

If main() calls pthread_exit() before other threads, the other threads continue to execute

Creating and terminating threads

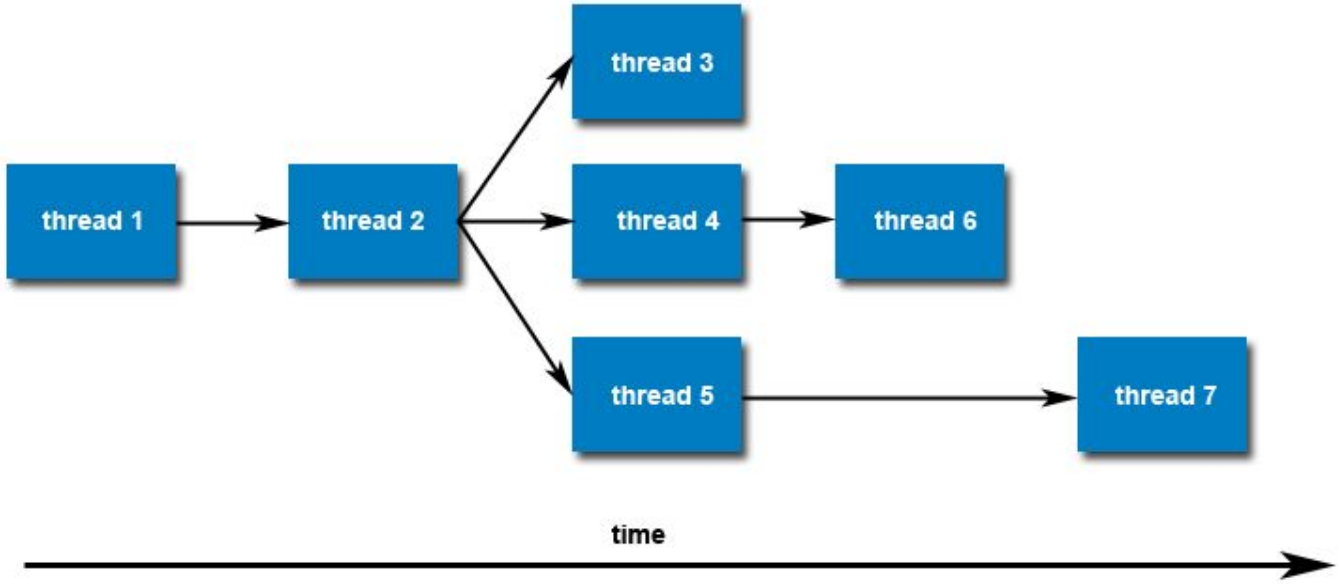
```c
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS     5

void *printHello(void *threadid){
   long tid;
   tid = (long)threadid;
   printf("Hello! thread #%ld!\n", tid);
   pthread_exit(NULL);
}
```

```c
int main (int argc, char *argv[]) {
   pthread_t threads[NUM_THREADS];
   int rc;
   long t;
   for(t = 0; t < NUM_THREADS; t++) {
      printf("In main: creating thread %ld\n", t);
      rc = pthread_create(&threads[t], NULL,
                  printHello, (void *)t);
      if (rc) {
         printf("ERROR; ret val:  %d\n", rc);
         exit(-1);
      }
   }
   /* Last thing that main() should do */
   pthread_exit(NULL);
}
```

Initially, your main() program comprises a single, default thread. All other threads must be explicitly created by the programmer.

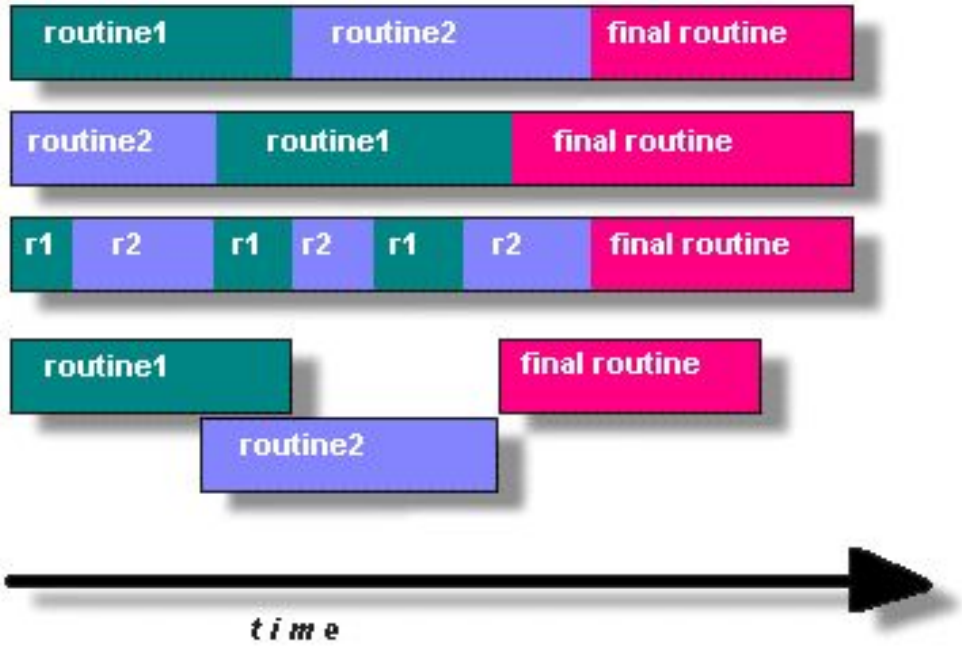
Once created, threads are peers, and may create other threads. There is no implied hierarchy or dependency between threads.



Designing threaded programs

For multi-thread programming

★ The program must be organized into discrete, independent tasks which can execute concurrently.



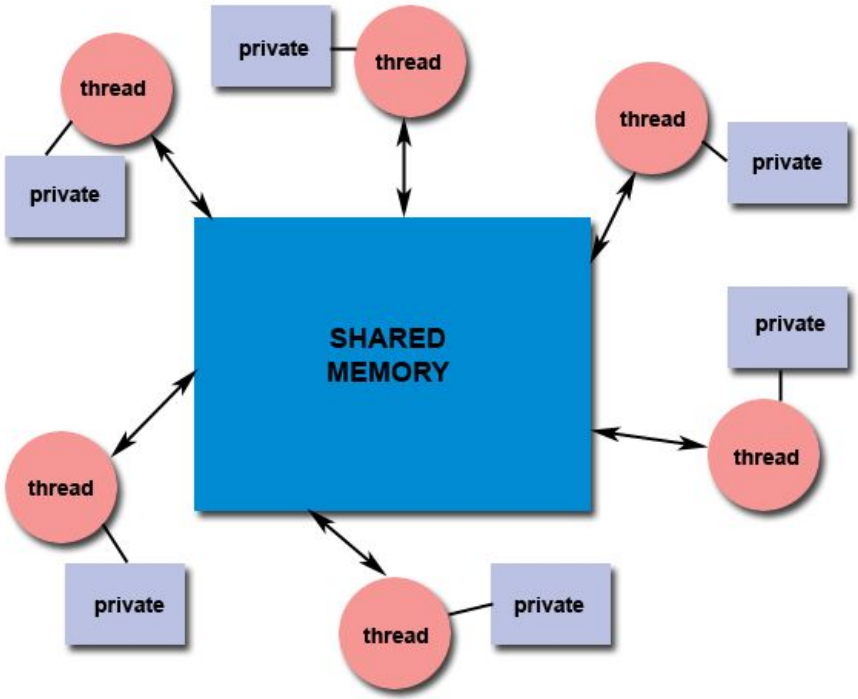
For example, if **routine1** and **routine2** can be interchanged, interleaved and/or overlapped in real time, they are candidates for threading.

Shared memory model

All threads have access to the same global, shared memory

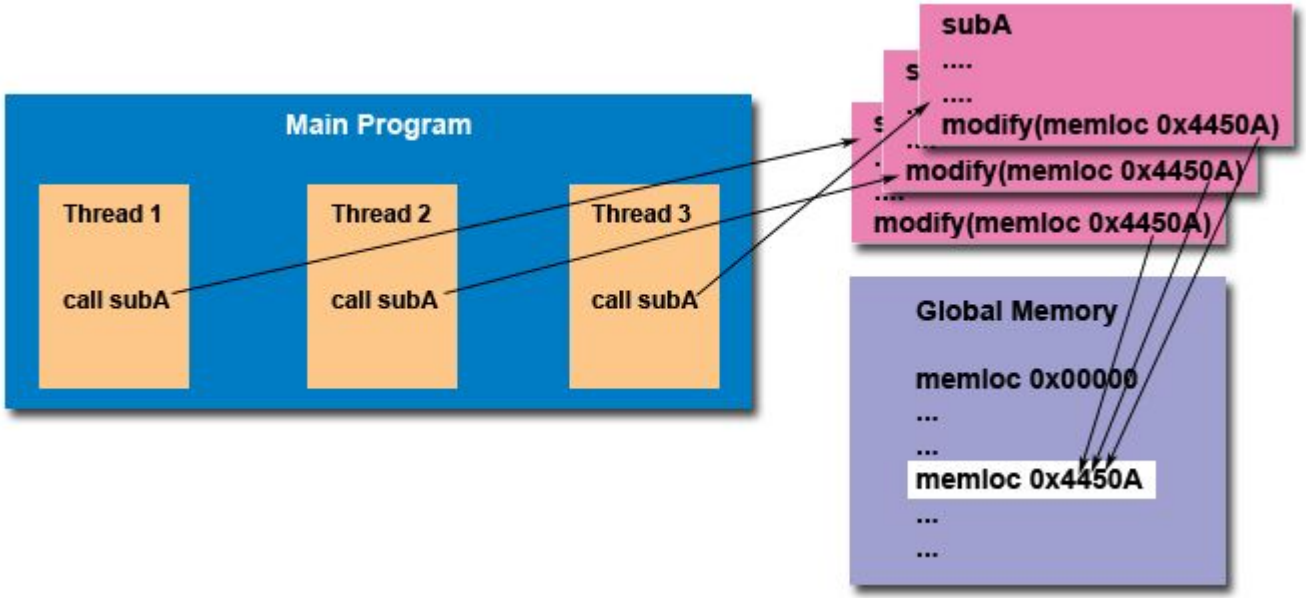
Threads also have their own private data

Programmers are responsible for synchronizing access (protecting) globally shared data.



Thread safeness

an application's ability to execute multiple threads simultaneously without "clobbering" shared data or creating "race" conditions



Passing arguments to threads

```
int taskids[NUM_THREADS];

for(t=0; t<NUM_THREADS; t++){
  taskids[t] = t;
  printf("Creating thread %ld\n", t);
  rc = pthread_create(&threads[t], NULL,printHello, (void*)&taskids[t]);
  ...
}
```

Passing multiple arguments

```
struct thread_data
thread_data_array[NUM_THREADS];

void *printHello(void *threadarg) {
  struct thread_data *my_data;
  ...
  my_data = (struct thread_data *) threadarg;
  taskid = my_data->thread_id;
  sum = my_data->sum;
  hello_msg = my_data->message;
  ...
}
```

```
int main (int argc, char *argv[]) {
  ...
  thread_data_array[t].thread_id = t;
  thread_data_array[t].sum = sum;
  thread_data_array[t].message = messages[t];

  rc = pthread_create(&threads[t], NULL,
          printHello,
         (void *) &thread_data_array[t]);
  ...
}
```

What is wrong with this code?

```
int rc;
int t;
for(t=0; t<NUM_THREADS; t++) {
  printf("Creating thread %ld\n", t);
  rc = pthread_create(&threads[t], NULL, printHello, (void *) &t);
  ...
}
```

Argument passing incorrect: It passes the address of variable `t`, which is shared memory space and visible to all threads. As the loop iterates, the value of this memory location changes, possibly before the created threads can access it.

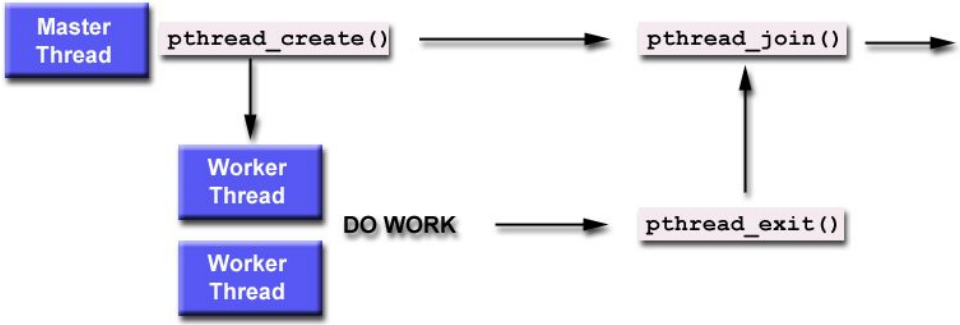
Joining threads (similar to “waitpid”)

"Joining" is one way to accomplish synchronization between threads.

```
#include <pthread.h>
int pthread_join(pthread_t threadid,
                 void **retval);
```

- The **pthread_join()** subroutine blocks the calling thread until the specified `threadid` thread terminates.
- If `retval` is not NULL, it points to a copy of terminated thread’s return value or the value specified in **pthread_exit()**.

Any thread in a process can use **pthread_join()** to join with any other thread in the process (there is no hierarchy).



Detaching Threads

- By default, a thread is joinable
  - when it terminates, another thread can obtain its return status using **pthread_join()**
- If we don't care about return value and want the system to cleanup the resources on termination, we mark thread as **detached.**
- Once the thread is detached, it is no longer possible to use pthread_join() to obtain its status

```
int pthread_detach(pthread_t thread);
```

- `thread:` target thread id
- Thread resources can be reclaimed on termination
- Returns 0 on success error code on failure
- this will explicitly detach a thread even though it is created as joinable.

Miscellaneous Routines

```
pthread_t pthread_self(void);
```

- returns the thread ID of the calling thread.

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

- compares the thread IDs t1 and t2.
- returns a non-zero value if t1 and t2 are equal; otherwise, zero shall be returned.

Threads-summary

- Exists within a process and uses the process resources
- Has its own independent flow of control as long as its parent-process exists and the OS supports it
- Duplicates only the essential resources it needs to be independently schedulable
- May share the process resources with other threads that act equally independently (and dependently)
- Dies if the parent process dies - or something similar
- Is "lightweight" because most of the overhead has already been accomplished through the creation of its process

- Because threads within the same process share resources:
  - Changes made by one thread to shared system resources (such as closing a file) will be seen by all other threads.
  - Two pointers having the same value point to the same data.
  - Reading and writing to the same memory locations is possible, and therefore requires explicit synchronization by the programmer.